

Exam templates — copy, paste, rename

Every `.java` file is self-contained: it has a **working dummy class at the bottom** so you can run it as is and see it green, then replace the dummy with your real class and rename. All of them were compiled and run in this project (JUnit 5.10, EasyMock 5.2, Spring 3.2). Package line says `de.tum.ise` — change it to the exam project's package.

The task says...	Use	What is inside
"implement the class / enum / method, then test it"	ImplementTemplate.java	enum (plain and with a value), result class, guards, compute→check→mutate, <code>Math.min</code> , cap
"write unit tests" (normal, other branch, failure, cap, exact boundary, invalid, exception)	TestTemplate.java	the seven tests every class needs, <code>@BeforeEach</code> , delta
exceptions, void methods, lists, object equality, <code>assertAll</code> , <code>@Nested</code> , skip	JUnitExtrasTemplate.java	<code>assertThrows</code> + message, state assertions, collections, <code>assertSame</code> , <code>assumeTrue</code>
a table of inputs/expected results, boundaries, "many cases"	ParameterizedTemplate.java	<code>@CsvSource</code> , <code>@ValueSource</code> , <code>@NullAndEmptySource</code> , <code>@MethodSource</code> , <code>@EnumSource</code>
"use a mock", manual style (<code>createMock</code>)	MockTemplate.java	create → record → replay → verify, <code>times</code> , void, throw, <code>anyString</code>
"annotate the test class", <code>@TestSubject</code> / <code>@Mock</code>	EasyMockAnnotationTemplate.java	the 3 injection rules, nice mock, capture, <code>andAnswer</code> , matchers, strict order, never-called
"implement the REST endpoints"	RestResourceTemplate.java	service over a list + resource with all 5 verbs, status codes, query param
"test the endpoints"	RestTestTemplate.java	<code>MockMvc</code> standalone, one test per status-code row, read-back after update/delete
"equivalence classes / boundary values"	BlackBoxTemplate.md	the three tables with <code><fill></code> cells, class-count check, L-1 / L / L+1 recipe
"write acceptance tests / Gherkin"	GherkinTemplate.md	Feature, Background, happy + failure scenario, Scenario Outline

Rename checklist (60 seconds)

1. **Copy names from the task character-for-character** — class, method, enum constants, `@PathVariable` names.
2. Replace the dummy classes at the bottom of the file; delete the dummy `// ----- DELETE these -----` block.
3. Fix the numbers: write the expected values **by hand from the task's formulas**, never by calling the SUT.
4. Run it. A test must call the method under test, assert the result **and** the state, and use a delta on doubles.
5. `git add . && git commit -m "..."` && `git push`.

The one question to ask of every test

If I deleted the body of the method I am testing, would this test fail?

If the answer is no, it scores zero (the grader runs your tests against a broken implementation).